

Praktikum 10: Storage Management & I/O Systems

Informasi Praktikum

- **Mata Kuliah:** Sistem Operasi
 - **Sistem Operasi:** Linux Ubuntu
 - **Bahasa Pemrograman:** C
 - **Dosen Pengampu:** Triawan Adi Cahyanto, M.Kom
 - **Asisten Dosen:** Atidhira Habibillah dan Taqiyyuddin
-

Tujuan Pembelajaran

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan dapat:

1. Memahami konsep dasar perangkat I/O dan cara kerjanya
 2. Mengimplementasikan operasi I/O menggunakan bahasa C
 3. Memahami interface antara aplikasi dan kernel untuk I/O
 4. Menganalisis performa operasi I/O
 5. Memahami alur transformasi request I/O ke hardware
-

Persiapan Praktikum

A. Tool yang Dibutuhkan

```
# Update sistem terlebih dahulu
sudo apt update

# Install compiler C dan tools pendukung
sudo apt install build-essential gcc make

# Install tools monitoring
sudo apt install sysstat iotop htop

# Install library development
sudo apt install libaio-dev

# Verifikasi instalasi
gcc --version
```

B. Persiapan Direktori Kerja

```
# Buat direktori untuk praktikum
mkdir -p ~/praktikum-io-systems
cd ~/praktikum-io-systems
```

```
# Buat subdirektori untuk setiap modul  
mkdir -p {hardware,interface,kernel,transform,streams,performance}
```

Topik 1: I/O Hardware

1.1 Pengenalan I/O Hardware

Konsep Dasar:

- I/O Hardware adalah perangkat keras yang menghubungkan CPU dengan perangkat eksternal
- Terdiri dari: Controller, Bus, Port, dan Device Driver
- Komunikasi menggunakan: Polling, Interrupt, dan DMA (Direct Memory Access)

1.2 Praktik: Melihat Informasi I/O Hardware

```
cd ~/praktikum-io-systems/hardware  
  
# 1. Lihat semua perangkat I/O yang terhubung  
lspci -v | less  
  
# 2. Lihat informasi USB devices  
lsusb -v  
  
# 3. Lihat informasi storage devices  
lsblk -f  
  
# 4. Lihat detail hardware dengan dmidecode  
sudo dmidecode -t memory  
sudo dmidecode -t processor  
  
# 5. Lihat interrupt yang digunakan  
cat /proc/interrupts  
  
# 6. Lihat DMA channels  
cat /proc/dma  
  
# 7. Lihat I/O ports  
cat /proc/ioports
```

Tugas 1.1: Dokumentasikan hasil output dari perintah di atas dan identifikasi:

- Jenis-jenis perangkat I/O yang terhubung
- IRQ (Interrupt Request) yang digunakan
- I/O port yang terpakai

1.3 Praktik: Monitoring I/O Hardware

Buat file `monitor_io.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

void read_proc_file(const char *filename) {
    FILE *fp = fopen(filename, "r");
    if (fp == NULL) {
        perror("Error opening file");
        return;
    }

    char buffer[256];
    printf("\n=== Content of %s ===\n", filename);
    while (fgets(buffer, sizeof(buffer), fp) != NULL) {
        printf("%s", buffer);
    }
    fclose(fp);
}

void display_io_stats() {
    printf("\n===== \n");
    printf("I/O HARDWARE MONITORING\n");
    printf("===== \n");

    // Tampilkan interrupt
    read_proc_file("/proc/interrupts");

    // Tampilkan statistik disk
    read_proc_file("/proc/diskstats");
}

int main() {
    printf("Starting I/O Hardware Monitor...\n");
    printf("Press Ctrl+C to stop\n");

    while(1) {
        display_io_stats();
        sleep(5); // Update setiap 5 detik
        system("clear");
    }

    return 0;
}
```

Compile dan Jalankan:

```
gcc -o monitor_io monitor_io.c
./monitor_io
```

Tugas 1.2: Amati perubahan statistik I/O selama 2 menit dan catat:

- Interrupt mana yang paling sering terjadi?
 - Disk mana yang paling aktif?
-

Topik 2: Application I/O Interface

2.1 Pengenalan Application I/O Interface

Konsep Dasar:

- Interface yang menyediakan abstraksi untuk aplikasi mengakses perangkat I/O
- Tipe device: Block devices (disk), Character devices (keyboard), Network devices
- System calls yang umum: open(), read(), write(), close(), ioctl()

2.2 Praktik: Block Device I/O

```
cd ~/praktikum-io-systems/interface
```

Buat file `block_io.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <time.h>

#define BUFFER_SIZE 4096

void write_file_example(const char *filename) {
    int fd;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_written;

    // Buka file dengan flag O_CREAT | O_WRONLY | O_TRUNC
    fd = open(filename, O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd == -1) {
        perror("Error opening file for writing");
        return;
    }

    // Isi buffer dengan data
    strcpy(buffer, "Ini adalah contoh data untuk Block I/O\n");

    // Tulis ke file
    bytes_written = write(fd, buffer, strlen(buffer));
    if (bytes_written == -1) {
        perror("Error writing to file");
    }
}
```

```
        close(fd);
        return;
    }

    printf("Successfully wrote %ld bytes to %s\n", bytes_written,
filename);

    // Sync data ke disk
    fsync(fd);

    close(fd);
}

void read_file_example(const char *filename) {
    int fd;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read;

    // Buka file untuk reading
    fd = open(filename, O_RDONLY);
    if (fd == -1) {
        perror("Error opening file for reading");
        return;
    }

    // Baca dari file
    bytes_read = read(fd, buffer, BUFFER_SIZE - 1);
    if (bytes_read == -1) {
        perror("Error reading from file");
        close(fd);
        return;
    }

    buffer[bytes_read] = '\0'; // Null terminate
    printf("Read %ld bytes: %s", bytes_read, buffer);

    close(fd);
}

void sequential_io_test(const char *filename, int num_operations) {
    int fd;
    char buffer[BUFFER_SIZE];
    clock_t start, end;
    double cpu_time_used;

    printf("\n=== Sequential I/O Test ===\n");

    // Write test
    fd = open(filename, O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd == -1) {
        perror("Error opening file");
        return;
    }
}
```

```
memset(buffer, 'A', BUFFER_SIZE);

start = clock();
for (int i = 0; i < num_operations; i++) {
    write(fd, buffer, BUFFER_SIZE);
}
fsync(fd);
end = clock();
close(fd);

cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC;
printf("Write: %d operations in %.4f seconds\n", num_operations,
cpu_time_used);
printf("Throughput: %.2f MB/s\n",
      (num_operations * BUFFER_SIZE / (1024.0 * 1024.0)) /
cpu_time_used);

// Read test
fd = open(filename, O_RDONLY);
if (fd == -1) {
    perror("Error opening file");
    return;
}

start = clock();
for (int i = 0; i < num_operations; i++) {
    read(fd, buffer, BUFFER_SIZE);
}
end = clock();
close(fd);

cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC;
printf("Read: %d operations in %.4f seconds\n", num_operations,
cpu_time_used);
printf("Throughput: %.2f MB/s\n",
      (num_operations * BUFFER_SIZE / (1024.0 * 1024.0)) /
cpu_time_used);
}

int main() {
    const char *test_file = "test_block_io.dat";

    printf("=== Application I/O Interface Demo ===\n\n");

    // Test basic write and read
    printf("1. Basic Write Operation:\n");
    write_file_example(test_file);

    printf("\n2. Basic Read Operation:\n");
    read_file_example(test_file);

    // Test sequential I/O performance
    sequential_io_test("test_sequential.dat", 1000);
```

```
// Cleanup
unlink(test_file);
unlink("test_sequential.dat");

return 0;
}
```

Compile dan Jalankan:

```
gcc -o block_io block_io.c
./block_io
```

2.3 Praktik: Character Device I/O

Buat file `char_io.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <termios.h>
#include <string.h>

void read_keyboard_unbuffered() {
    struct termios old_term, new_term;
    char ch;

    // Dapatkan setting terminal saat ini
    tcgetattr(STDIN_FILENO, &old_term);
    new_term = old_term;

    // Disable canonical mode dan echo
    new_term.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &new_term);

    printf("Press keys (press 'q' to quit):\n");

    while (1) {
        ch = getchar();
        if (ch == 'q') break;
        printf("You pressed: %c (ASCII: %d)\n", ch, ch);
    }

    // Restore terminal settings
    tcsetattr(STDIN_FILENO, TCSANOW, &old_term);
}

void read_random_device() {
    int fd;
```

```
    unsigned char buffer[16];
    ssize_t bytes_read;

    printf("\n=== Reading from /dev/urandom ===\n");

    fd = open("/dev/urandom", O_RDONLY);
    if (fd == -1) {
        perror("Error opening /dev/urandom");
        return;
    }

    bytes_read = read(fd, buffer, sizeof(buffer));
    if (bytes_read == -1) {
        perror("Error reading");
        close(fd);
        return;
    }

    printf("Random bytes: ");
    for (int i = 0; i < bytes_read; i++) {
        printf("%02x ", buffer[i]);
    }
    printf("\n");

    close(fd);
}

void read_null_device() {
    int fd;
    char buffer[100];
    ssize_t bytes_read;

    printf("\n=== Reading from /dev/null ===\n");

    fd = open("/dev/null", O_RDONLY);
    if (fd == -1) {
        perror("Error opening /dev/null");
        return;
    }

    bytes_read = read(fd, buffer, sizeof(buffer));
    printf("Bytes read from /dev/null: %ld (always returns 0)\n",
bytes_read);

    close(fd);

    // Write to /dev/null
    fd = open("/dev/null", O_WRONLY);
    write(fd, "This data disappears", 20);
    printf("Data written to /dev/null (data is discarded)\n");
    close(fd);
}

int main() {
```



```
printf("=== Character Device I/O Demo ===\n\n");

read_random_device();
read_null_device();

printf("\n=== Keyboard Input Demo ===\n");
read_keyboard_unbuffered();

return 0;
}
```

Compile dan Jalankan:

```
gcc -o char_io char_io.c
./char_io
```

Tugas 2.1:

1. Jalankan kedua program dan catat perbedaan antara block device dan character device
2. Modifikasi `block_io.c` untuk menggunakan buffer dengan ukuran berbeda (512 bytes, 4KB, 64KB) dan bandingkan performanya

Topik 3: Kernel I/O Subsystem

3.1 Pengenalan Kernel I/O Subsystem

Konsep Dasar:

- Buffering: menyimpan data sementara selama transfer
- Caching: menyimpan salinan data untuk akses cepat
- Spooling: menyimpan output untuk device yang tidak bisa menerima interleaved data
- Device reservation: mengalokasikan device secara eksklusif
- Error handling: menangani error dari hardware

3.2 Praktik: Buffering dan Caching

```
cd ~/praktikum-io-systems/kernel
```

Buat file `buffering.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>
```

```

#include <fcntl.h>

#define DATA_SIZE 1048576 // 1 MB

void test_unbuffered_io(const char *filename) {
    int fd;
    char *data;
    clock_t start, end;
    double time_used;

    data = malloc(DATA_SIZE);
    memset(data, 'A', DATA_SIZE);

    printf("\n=== Unbuffered I/O Test ===\n");

    // Open dengan O_DIRECT untuk bypass cache (unbuffered)
    // Note: O_DIRECT memerlukan alignment khusus, jadi kita gunakan
    O_SYNC
    fd = open(filename, O_CREAT | O_WRONLY | O_SYNC | O_TRUNC, 0644);
    if (fd == -1) {
        perror("Error opening file");
        free(data);
        return;
    }

    start = clock();

    // Tulis dalam chunk kecil
    for (int i = 0; i < DATA_SIZE; i += 1024) {
        write(fd, data + i, 1024);
    }

    end = clock();
    close(fd);

    time_used = ((double) (end - start)) / CLOCKS_PER_SEC;
    printf("Time taken (unbuffered): %.4f seconds\n", time_used);
    printf("Throughput: %.2f MB/s\n", (DATA_SIZE / (1024.0 * 1024.0)) /
time_used);

    free(data);
}

void test_buffered_io(const char *filename) {
    FILE *fp;
    char *data;
    clock_t start, end;
    double time_used;

    data = malloc(DATA_SIZE);
    memset(data, 'A', DATA_SIZE);

    printf("\n=== Buffered I/O Test ===\n");

```

```

    fp = fopen(filename, "w");
    if (fp == NULL) {
        perror("Error opening file");
        free(data);
        return;
    }

    // Set buffer size
    setvbuf(fp, NULL, _IOFBF, 8192);

    start = clock();

    // Tulis dalam chunk kecil
    for (int i = 0; i < DATA_SIZE; i += 1024) {
        fwrite(data + i, 1, 1024, fp);
    }
    fflush(fp);

    end = clock();
    fclose(fp);

    time_used = ((double) (end - start)) / CLOCKS_PER_SEC;
    printf("Time taken (buffered): %.4f seconds\n", time_used);
    printf("Throughput: %.2f MB/s\n", (DATA_SIZE / (1024.0 * 1024.0)) /
time_used);

    free(data);
}

void demonstrate_cache_effect() {
    int fd;
    char buffer[4096];
    clock_t start, end;
    double first_read, second_read;

    printf("\n=== Cache Effect Demonstration ===\n");

    const char *filename = "cache_test.dat";

    // Buat file test
    fd = open(filename, O_CREAT | O_WRONLY | O_TRUNC, 0644);
    for (int i = 0; i < 1000; i++) {
        write(fd, buffer, sizeof(buffer));
    }
    close(fd);

    // Clear cache (perlu sudo)
    printf("Attempting to clear cache (may require sudo)...\n");
    system("sync");
    system("sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches' 2>/dev/null");

    // First read (from disk)
    fd = open(filename, O_RDONLY);
    start = clock();

```

```

while (read(fd, buffer, sizeof(buffer)) > 0);
end = clock();
close(fd);
first_read = ((double) (end - start)) / CLOCKS_PER_SEC;

// Second read (from cache)
fd = open(filename, O_RDONLY);
start = clock();
while (read(fd, buffer, sizeof(buffer)) > 0);
end = clock();
close(fd);
second_read = ((double) (end - start)) / CLOCKS_PER_SEC;

printf("First read (from disk): %.4f seconds\n", first_read);
printf("Second read (from cache): %.4f seconds\n", second_read);
printf("Speedup: %.2fx\n", first_read / second_read);

unlink(filename);
}

int main() {
    printf("=== Kernel I/O Subsystem: Buffering & Caching ===\n");

    test_unbuffered_io("test_unbuffered.dat");
    test_buffered_io("test_buffered.dat");
    demonstrate_cache_effect();

    // Cleanup
    unlink("test_unbuffered.dat");
    unlink("test_buffered.dat");

    return 0;
}

```

Compile dan Jalankan:

```

gcc -o buffering buffering.c
./buffering

```

3.3 Praktik: I/O Scheduling

Buat file `io_scheduling.c`:

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/time.h>
#include <string.h>

```

```

#define NUM_REQUESTS 100
#define BLOCK_SIZE 4096

void check_io_scheduler() {
    FILE *fp;
    char buffer[256];

    printf("=== Current I/O Scheduler ===\n");

    // Cek scheduler untuk sda
    fp = fopen("/sys/block/sda/queue/scheduler", "r");
    if (fp != NULL) {
        if (fgets(buffer, sizeof(buffer), fp) != NULL) {
            printf("SDA Scheduler: %s", buffer);
        }
        fclose(fp);
    }

    // Cek scheduler parameters
    fp = fopen("/sys/block/sda/queue/nr_requests", "r");
    if (fp != NULL) {
        if (fgets(buffer, sizeof(buffer), fp) != NULL) {
            printf("Queue depth: %s", buffer);
        }
        fclose(fp);
    }
}

void random_io_pattern(const char *filename) {
    int fd;
    char buffer[BLOCK_SIZE];
    struct timeval start, end;
    long seconds, useconds;
    double duration;

    printf("\n=== Random I/O Pattern ===\n");

    fd = open(filename, O_CREAT | O_RDWR | O_TRUNC, 0644);
    if (fd == -1) {
        perror("Error opening file");
        return;
    }

    // Buat file dengan ukuran tertentu
    memset(buffer, 'X', BLOCK_SIZE);
    for (int i = 0; i < 1000; i++) {
        write(fd, buffer, BLOCK_SIZE);
    }

    gettimeofday(&start, NULL);

    // Random seeks and reads
    for (int i = 0; i < NUM_REQUESTS; i++) {

```

```
        off_t offset = (rand() % 1000) * BLOCK_SIZE;
        lseek(fd, offset, SEEK_SET);
        read(fd, buffer, BLOCK_SIZE);
    }

    gettimeofday(&end, NULL);

    seconds = end.tv_sec - start.tv_sec;
    useconds = end.tv_usec - start.tv_usec;
    duration = seconds + useconds / 1000000.0;

    printf("Random I/O: %d requests in %.4f seconds\n", NUM_REQUESTS,
duration);
    printf("IOPS: %.2f\n", NUM_REQUESTS / duration);

    close(fd);
}

void sequential_io_pattern(const char *filename) {
    int fd;
    char buffer[BLOCK_SIZE];
    struct timeval start, end;
    long seconds, useconds;
    double duration;

    printf("\n=== Sequential I/O Pattern ===\n");

    fd = open(filename, O_RDONLY);
    if (fd == -1) {
        perror("Error opening file");
        return;
    }

    gettimeofday(&start, NULL);

    // Sequential reads
    for (int i = 0; i < NUM_REQUESTS; i++) {
        read(fd, buffer, BLOCK_SIZE);
    }

    gettimeofday(&end, NULL);

    seconds = end.tv_sec - start.tv_sec;
    useconds = end.tv_usec - start.tv_usec;
    duration = seconds + useconds / 1000000.0;

    printf("Sequential I/O: %d requests in %.4f seconds\n", NUM_REQUESTS,
duration);
    printf("IOPS: %.2f\n", NUM_REQUESTS / duration);

    close(fd);
}

int main() {
```

```
const char *test_file = "io_schedule_test.dat";

printf("=== I/O Scheduling Demo ===\n\n");

check_io_scheduler();
random_io_pattern(test_file);
sequential_io_pattern(test_file);

printf("\nNote: Sequential I/O should be faster due to I/O scheduler
optimization\n");

unlink(test_file);
return 0;
}
```

Compile dan Jalankan:

```
gcc -o io_scheduling io_scheduling.c
./io_scheduling
```

Tugas 3.1:

1. Jelaskan mengapa buffered I/O lebih cepat dari unbuffered I/O
2. Jelaskan mengapa second read lebih cepat (cache effect)
3. Bandingkan IOPS antara random dan sequential I/O

Topik 4: Transforming I/O Requests to Hardware Operations

4.1 Pengenalan Transformasi I/O Request

Konsep Dasar:

- Layering dalam I/O: Application → System Call → Kernel I/O → Device Driver → Hardware
- Buffer management dan data transfer
- Interrupt handling dan completion

4.2 Praktik: Tracing I/O Operations

```
cd ~/praktikum-io-systems/transform
```

Buat file `trace_io.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
```

```

#include <sys/types.h>
#include <sys/stat.h>
#include <string.h>
#include <errno.h>
#include <time.h>

void log_operation(const char *operation, const char *details) {
    time_t now;
    char timestamp[26];

    time(&now);
    ctime_r(&now, timestamp);
    timestamp[24] = '\0'; // Remove newline

    printf("[%s] %s: %s\n", timestamp, operation, details);
}

void trace_file_operations(const char *filename) {
    int fd;
    char buffer[1024];
    ssize_t bytes;
    struct stat file_stat;
    char details[256];

    printf("\n=== Tracing I/O Operations ===\n\n");

    // STEP 1: Application layer – open() system call
    log_operation("APP_LAYER", "Calling open() system call");
    snprintf(details, sizeof(details), "open(\"%s\",
O_CREAT|O_RDWR|O_TRUNC, 0644)", filename);
    log_operation("SYSCALL", details);

    fd = open(filename, O_CREAT | O_RDWR | O_TRUNC, 0644);
    if (fd == -1) {
        snprintf(details, sizeof(details), "open() failed: %s",
strerror(errno));
        log_operation("ERROR", details);
        return;
    }

    snprintf(details, sizeof(details), "File descriptor allocated: fd=%d",
fd);
    log_operation("KERNEL", details);

    // Get file information
    if (fstat(fd, &file_stat) == 0) {
        snprintf(details, sizeof(details), "Inode: %ld, Device: %ld",
            (long)file_stat.st_ino, (long)file_stat.st_dev);
        log_operation("FILESYSTEM", details);
    }

    // STEP 2: Write operation
    strcpy(buffer, "Hello, I/O System!");
    log_operation("APP_LAYER", "Calling write() system call");

```



```

    snprintf(details, sizeof(details), "write(fd=%d, buffer, %ld)", fd,
strlen(buffer));
    log_operation("SYSCALL", details);

    bytes = write(fd, buffer, strlen(buffer));
    if (bytes == -1) {
        snprintf(details, sizeof(details), "write() failed: %s",
strerror(errno));
        log_operation("ERROR", details);
    } else {
        snprintf(details, sizeof(details), "Bytes written to buffer: %ld",
bytes);
        log_operation("KERNEL", details);
        log_operation("BUFFER", "Data copied to kernel buffer");
    }

    // STEP 3: Sync to disk
    log_operation("APP_LAYER", "Calling fsync() to flush to disk");
    if (fsync(fd) == 0) {
        log_operation("KERNEL", "Initiating disk write");
        log_operation("DEVICE_DRIVER", "Preparing DMA transfer");
        log_operation("HARDWARE", "Data written to physical disk");
        log_operation("INTERRUPT", "Disk controller sends completion
interrupt");
        log_operation("KERNEL", "I/O completion handled");
    }

    // STEP 4: Seek operation
    log_operation("APP_LAYER", "Calling lseek() to beginning");
    if (lseek(fd, 0, SEEK_SET) != -1) {
        log_operation("KERNEL", "File pointer repositioned to offset 0");
    }

    // STEP 5: Read operation
    memset(buffer, 0, sizeof(buffer));
    log_operation("APP_LAYER", "Calling read() system call");
    snprintf(details, sizeof(details), "read(fd=%d, buffer, 1024)", fd);
    log_operation("SYSCALL", details);

    bytes = read(fd, buffer, sizeof(buffer));
    if (bytes == -1) {
        snprintf(details, sizeof(details), "read() failed: %s",
strerror(errno));
        log_operation("ERROR", details);
    } else {
        log_operation("KERNEL", "Checking page cache");
        log_operation("CACHE", "Data found in cache (hot read)");
        snprintf(details, sizeof(details), "Bytes read: %ld, Content:
\\\"%s\\\"\"", bytes, buffer);
        log_operation("APP_LAYER", details);
    }

    // STEP 6: Close operation
    log_operation("APP_LAYER", "Calling close() system call");

```

```

    if (close(fd) == 0) {
        log_operation("KERNEL", "File descriptor released");
        log_operation("FILESYSTEM", "File metadata updated");
    }

    printf("\n=== I/O Request Transformation Complete ===\n");
}

void demonstrate_layers() {
    printf("\n=== I/O Layers Explanation ===\n\n");

    printf("Layer 1 - APPLICATION LAYER:\n");
    printf(" - User program calls standard I/O functions\n");
    printf(" - Example: fopen(), fwrite(), fread()\n\n");

    printf("Layer 2 - SYSTEM CALL INTERFACE:\n");
    printf(" - Translates library calls to system calls\n");
    printf(" - Example: open(), write(), read(), close()\n\n");

    printf("Layer 3 - KERNEL I/O SUBSYSTEM:\n");
    printf(" - Buffer management and caching\n");
    printf(" - I/O scheduling and optimization\n\n");

    printf("Layer 4 - DEVICE DRIVER:\n");
    printf(" - Converts generic I/O to device-specific commands\n");
    printf(" - Manages device registers and DMA\n\n");

    printf("Layer 5 - HARDWARE CONTROLLER:\n");
    printf(" - Physical device interface\n");
    printf(" - Generates interrupts on completion\n\n");
}

int main() {
    const char *test_file = "trace_test.txt";

    demonstrate_layers();
    trace_file_operations(test_file);

    // Cleanup
    unlink(test_file);

    return 0;
}

```

Compile dan Jalankan:

```

gcc -o trace_io trace_io.c
./trace_io

```

4.3 Praktik: System Call Monitoring dengan strace

```
# Buat program sederhana untuk di-trace
cat > simple_io.c << 'EOF'
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("test.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    write(fd, "Hello World\n", 12);
    close(fd);
    return 0;
}
EOF

gcc -o simple_io simple_io.c

# Trace system calls
echo "=== Tracing System Calls ==="
strace -c ./simple_io # Summary
strace -tt ./simple_io 2>&1 | grep -E "open|write|close" # Detailed

# Trace dengan filter tertentu
strace -e trace=open,read,write,close ./simple_io

# Trace dengan timing
strace -T ./simple_io 2>&1 | grep -E "open|write|close"
```

4.4 Praktik: Monitoring I/O dengan iotop dan iostat

```
# Terminal 1: Jalankan program I/O intensive
cd ~/praktikum-io-systems/transform

cat > io_intensive.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;
    char buffer[4096];

    printf("Starting I/O intensive operations...\n");

    for (int i = 0; i < 10000; i++) {
        fd = open("temp_io.dat", O_CREAT | O_WRONLY | O_TRUNC, 0644);
        write(fd, buffer, sizeof(buffer));
        fsync(fd);
        close(fd);
    }
}
```

```

        if (i % 1000 == 0) {
            printf("Progress: %d/10000\n", i);
        }
    }

    unlink("temp_io.dat");
    printf("Completed!\n");
    return 0;
}
EOF

gcc -o io_intensive io_intensive.c

# Terminal 2: Monitor dengan iostat
# iostat -x 1 # Update setiap 1 detik

# Terminal 3: Monitor dengan iotop (perlu sudo)
# sudo iotop -o # Hanya tampilkan proses dengan I/O aktif

```

Tugas 4.1:

1. Jalankan `trace_io` dan analisis setiap layer yang dilalui
2. Gunakan `strace` untuk melihat system call yang dipanggil
3. Identifikasi waktu yang dihabiskan di setiap layer
4. Jelaskan peran interrupt dalam menyelesaikan I/O operation

Topik 5: STREAMS

5.1 Pengenalan STREAMS

Konsep Dasar:

- STREAMS adalah framework untuk character I/O di UNIX
- Terdiri dari stream head, modules, dan driver
- Mendukung full-duplex communication
- Digunakan untuk networking, terminal I/O, dan pipes

5.2 Praktik: Pipes dan STREAMS

```
cd ~/praktikum-io-systems/streams
```

Buat file `pipe_demo.c`:

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

```

```
#include <sys/wait.h>

void simple_pipe_demo() {
    int pipefd[2];
    pid_t pid;
    char write_msg[] = "Hello from parent process!";
    char read_msg[100];

    printf("=== Simple Pipe Demo ===\n");

    // Buat pipe
    if (pipe(pipefd) == -1) {
        perror("pipe");
        return;
    }

    printf("Pipe created: read_fd=%d, write_fd=%d\n", pipefd[0],
pipefd[1]);

    pid = fork();

    if (pid == -1) {
        perror("fork");
        return;
    }

    if (pid == 0) {
        // Child process - reader
        close(pipefd[1]); // Close write end

        printf("Child: Waiting for data...\n");
        ssize_t bytes = read(pipefd[0], read_msg, sizeof(read_msg));

        if (bytes > 0) {
            read_msg[bytes] = '\0';
            printf("Child: Received %ld bytes: '%s'\n", bytes, read_msg);
        }

        close(pipefd[0]);
        exit(0);
    } else {
        // Parent process - writer
        close(pipefd[0]); // Close read end

        printf("Parent: Writing to pipe...\n");
        write(pipefd[1], write_msg, strlen(write_msg));

        printf("Parent: Data written, closing pipe\n");
        close(pipefd[1]);

        wait(NULL); // Wait for child
        printf("Parent: Child process completed\n");
    }
}
```

```
}

void bidirectional_pipe_demo() {
    int pipe1[2], pipe2[2];
    pid_t pid;
    char parent_msg[] = "Message from parent";
    char child_msg[] = "Message from child";
    char buffer[100];

    printf("\n=== Bidirectional Communication Demo ===\n");

    // Buat dua pipes untuk komunikasi dua arah
    if (pipe(pipe1) == -1 || pipe(pipe2) == -1) {
        perror("pipe");
        return;
    }

    printf("Two pipes created for bidirectional communication\n");

    pid = fork();

    if (pid == 0) {
        // Child process
        close(pipe1[1]); // Close write end of pipe1
        close(pipe2[0]); // Close read end of pipe2

        // Read from parent
        read(pipe1[0], buffer, sizeof(buffer));
        printf("Child received: %s\n", buffer);

        // Write to parent
        write(pipe2[1], child_msg, strlen(child_msg));
        printf("Child sent response\n");

        close(pipe1[0]);
        close(pipe2[1]);
        exit(0);
    } else {
        // Parent process
        close(pipe1[0]); // Close read end of pipe1
        close(pipe2[1]); // Close write end of pipe2

        // Write to child
        write(pipe1[1], parent_msg, strlen(parent_msg));
        printf("Parent sent: %s\n", parent_msg);

        // Read from child
        read(pipe2[0], buffer, sizeof(buffer));
        printf("Parent received: %s\n", buffer);

        close(pipe1[1]);
        close(pipe2[0]);
        wait(NULL);
    }
}
```

```

    }
}

void pipe_buffer_size_demo() {
    int pipefd[2];
    long pipe_size;

    printf("\n=== Pipe Buffer Size Demo ===\n");

    if (pipe(pipefd) == -1) {
        perror("pipe");
        return;
    }

    // Get pipe buffer size (Linux specific)
#ifdef __linux__
    pipe_size = fpathconf(pipefd[0], _PC_PIPE_BUF);
    printf("PIPE_BUF (atomic write size): %ld bytes\n", pipe_size);

    // Try to get pipe capacity (Linux 2.6.35+)
    FILE *fp = fopen("/proc/sys/fs/pipe-max-size", "r");
    if (fp) {
        long max_size;
        fscanf(fp, "%ld", &max_size);
        printf("Maximum pipe size: %ld bytes\n", max_size);
        fclose(fp);
    }
#endif

    close(pipefd[0]);
    close(pipefd[1]);
}

int main() {
    printf("=== STREAMS and Pipes Demo ===\n\n");

    simple_pipe_demo();
    bidirectional_pipe_demo();
    pipe_buffer_size_demo();

    return 0;
}

```

Compile dan Jalankan:

```

gcc -o pipe_demo pipe_demo.c
./pipe_demo

```

5.3 Praktik: Named Pipes (FIFO)

Buat file `fifo_writer.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>
#include <string.h>

int main() {
    const char *fifo_path = "/tmp/my_fifo";
    char message[100];
    int fd;
    int counter = 0;

    // Buat named pipe (FIFO)
    mkfifo(fifo_path, 0666);

    printf("FIFO Writer Started\n");
    printf("Opening FIFO for writing...\n");

    fd = open(fifo_path, O_WRONLY);
    printf("FIFO opened, ready to send messages\n");

    while (counter < 5) {
        snprintf(message, sizeof(message), "Message #%d from writer",
counter + 1);
        write(fd, message, strlen(message) + 1);
        printf("Sent: %s\n", message);

        sleep(2);
        counter++;
    }

    close(fd);
    printf("Writer finished\n");

    return 0;
}
```

Buat file `fifo_reader.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>
```



```

int main() {
    const char *fifo_path = "/tmp/my_fifo";
    char buffer[100];
    int fd;
    ssize_t bytes;

    printf("FIFO Reader Started\n");
    printf("Opening FIFO for reading...\n");

    fd = open(fifo_path, O_RDONLY);
    printf("FIFO opened, waiting for messages...\n\n");

    while ((bytes = read(fd, buffer, sizeof(buffer))) > 0) {
        printf("Received: %s\n", buffer);
    }

    close(fd);
    unlink(fifo_path);
    printf("\nReader finished\n");

    return 0;
}

```

Compile dan Jalankan:

```

gcc -o fifo_writer fifo_writer.c
gcc -o fifo_reader fifo_reader.c

# Terminal 1: Jalankan reader terlebih dahulu
./fifo_reader

# Terminal 2: Jalankan writer
./fifo_writer

```

5.4 Praktik: Stream Processing dengan Filter

Buat file `stream_filter.c`:

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <ctype.h>

void demonstrate_stream_processing() {
    int pipe_in[2], pipe_out[2];
    pid_t pid1, pid2;
    char input[] = "hello world from stream processing";
    char buffer[100];
}

```

```
printf("=== Stream Processing Pipeline Demo ===\n");
printf("Input: \"%s\"\n", input);
printf("Pipeline: Input → Uppercase Filter → Reverse Filter →
Output\n\n");

pipe(pipe_in);
pipe(pipe_out);

// Process 1: Uppercase filter
pid1 = fork();
if (pid1 == 0) {
    close(pipe_in[1]); // Close write end of input pipe
    close(pipe_out[0]); // Close read end of output pipe

    ssize_t n = read(pipe_in[0], buffer, sizeof(buffer));

    // Convert to uppercase
    for (int i = 0; i < n; i++) {
        buffer[i] = toupper(buffer[i]);
    }

    write(pipe_out[1], buffer, n);

    close(pipe_in[0]);
    close(pipe_out[1]);
    exit(0);
}

// Parent process: Write input
close(pipe_in[0]);
write(pipe_in[1], input, strlen(input));
close(pipe_in[1]);

// Wait and read output
close(pipe_out[1]);
ssize_t n = read(pipe_out[0], buffer, sizeof(buffer));
buffer[n] = '\0';

printf("After uppercase filter: \"%s\"\n", buffer);

// Reverse the string
for (int i = 0; i < n/2; i++) {
    char temp = buffer[i];
    buffer[i] = buffer[n-1-i];
    buffer[n-1-i] = temp;
}

printf("After reverse filter: \"%s\"\n", buffer);

close(pipe_out[0]);
wait(NULL);
}
```

```
void shell_pipeline_demo() {
    printf("\n=== Shell Pipeline Demonstration ===\n");
    printf("Executing: ls -l | grep .c | wc -l\n\n");

    // Demonstrasi bagaimana shell melakukan piping
    system("ls -l *.c 2>/dev/null | wc -l");
}

int main() {
    demonstrate_stream_processing();
    shell_pipeline_demo();

    return 0;
}
```

Compile dan Jalankan:

```
gcc -o stream_filter stream_filter.c
./stream_filter
```

Tugas 5.1:

1. Modifikasi `pipe_demo.c` untuk mengirim 10 message berbeda
2. Buat program FIFO yang mengirim struct data (bukan hanya string)
3. Buat pipeline dengan 3 filter: lowercase → remove spaces → count words
4. Jelaskan perbedaan antara pipe dan named pipe (FIFO)

Topik 6: Performance Analysis

6.1 Pengenalan Performance Analysis

Konsep Dasar:

- Metrics: Throughput, Latency, IOPS, Bandwidth
- Bottleneck: CPU, Memory, I/O, Network
- Optimization: Caching, Buffering, Asynchronous I/O

6.2 Praktik: I/O Performance Benchmarking

```
cd ~/praktikum-io-systems/performance
```

Buat file `io_benchmark.c`:

```
#include <stdio.h>
#include <stdlib.h>
```

```

#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <sys/time.h>
#include <sys/stat.h>

#define MB (1024 * 1024)

typedef struct {
    double duration;
    double throughput_mb_s;
    double iops;
    long total_bytes;
} BenchmarkResult;

void print_result(const char *test_name, BenchmarkResult result) {
    printf("\n--- %s ---\n", test_name);
    printf("Duration: %.4f seconds\n", result.duration);
    printf("Total Data: %.2f MB\n", result.total_bytes / (double)MB);
    printf("Throughput: %.2f MB/s\n", result.throughput_mb_s);
    printf("IOPS: %.2f\n", result.iops);
}

BenchmarkResult benchmark_sequential_write(const char *filename, size_t
block_size, int num_blocks) {
    int fd;
    char *buffer;
    struct timeval start, end;
    BenchmarkResult result = {0};

    buffer = malloc(block_size);
    memset(buffer, 'A', block_size);

    fd = open(filename, O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd == -1) {
        perror("open");
        free(buffer);
        return result;
    }

    gettimeofday(&start, NULL);

    for (int i = 0; i < num_blocks; i++) {
        write(fd, buffer, block_size);
    }
    fsync(fd);

    gettimeofday(&end, NULL);
    close(fd);

    result.duration = (end.tv_sec - start.tv_sec) +
        (end.tv_usec - start.tv_usec) / 1000000.0;
    result.total_bytes = (long)block_size * num_blocks;
    result.throughput_mb_s = (result.total_bytes / (double)MB) /

```

```

    result.duration;
    result.iops = num_blocks / result.duration;

    free(buffer);
    return result;
}

BenchmarkResult benchmark_sequential_read(const char *filename, size_t
block_size) {
    int fd;
    char *buffer;
    struct timeval start, end;
    BenchmarkResult result = {0};
    ssize_t bytes_read;
    int num_blocks = 0;

    buffer = malloc(block_size);

    fd = open(filename, O_RDONLY);
    if (fd == -1) {
        perror("open");
        free(buffer);
        return result;
    }

    gettimeofday(&start, NULL);

    while ((bytes_read = read(fd, buffer, block_size)) > 0) {
        result.total_bytes += bytes_read;
        num_blocks++;
    }

    gettimeofday(&end, NULL);
    close(fd);

    result.duration = (end.tv_sec - start.tv_sec) +
        (end.tv_usec - start.tv_usec) / 1000000.0;
    result.throughput_mb_s = (result.total_bytes / (double)MB) /
result.duration;
    result.iops = num_blocks / result.duration;

    free(buffer);
    return result;
}

BenchmarkResult benchmark_random_write(const char *filename, size_t
block_size, int num_ops) {
    int fd;
    char *buffer;
    struct timeval start, end;
    BenchmarkResult result = {0};

    buffer = malloc(block_size);
    memset(buffer, 'R', block_size);

```

```

// Buat file dengan ukuran tertentu terlebih dahulu
fd = open(filename, O_CREAT | O_RDWR | O_TRUNC, 0644);
if (fd == -1) {
    perror("open");
    free(buffer);
    return result;
}

// Pre-allocate file
for (int i = 0; i < num_ops * 2; i++) {
    write(fd, buffer, block_size);
}

gettimeofday(&start, NULL);

// Random writes
for (int i = 0; i < num_ops; i++) {
    off_t offset = (rand() % (num_ops * 2)) * block_size;
    lseek(fd, offset, SEEK_SET);
    write(fd, buffer, block_size);
    result.total_bytes += block_size;
}
fsync(fd);

gettimeofday(&end, NULL);
close(fd);

result.duration = (end.tv_sec - start.tv_sec) +
                  (end.tv_usec - start.tv_usec) / 1000000.0;
result.throughput_mb_s = (result.total_bytes / (double)MB) /
result.duration;
result.iops = num_ops / result.duration;

free(buffer);
return result;
}

BenchmarkResult benchmark_random_read(const char *filename, size_t
block_size, int num_ops) {
    int fd;
    char *buffer;
    struct timeval start, end;
    struct stat st;
    BenchmarkResult result = {0};

    buffer = malloc(block_size);

    fd = open(filename, O_RDONLY);
    if (fd == -1) {
        perror("open");
        free(buffer);
        return result;
    }

```

```

    fstat(fd, &st);
    off_t file_size = st.st_size;
    int max_blocks = file_size / block_size;

    gettimeofday(&start, NULL);

    for (int i = 0; i < num_ops; i++) {
        off_t offset = (rand() % max_blocks) * block_size;
        lseek(fd, offset, SEEK_SET);
        read(fd, buffer, block_size);
        result.total_bytes += block_size;
    }

    gettimeofday(&end, NULL);
    close(fd);

    result.duration = (end.tv_sec - start.tv_sec) +
                      (end.tv_usec - start.tv_usec) / 1000000.0;
    result.throughput_mb_s = (result.total_bytes / (double)MB) /
result.duration;
    result.iops = num_ops / result.duration;

    free(buffer);
    return result;
}

void compare_block_sizes() {
    const char *filename = "blocksize_test.dat";
    size_t block_sizes[] = {512, 4096, 65536, 1048576}; // 512B, 4KB,
64KB, 1MB
    int num_sizes = sizeof(block_sizes) / sizeof(block_sizes[0]);

    printf("\n===== \n");
    printf("BLOCK SIZE COMPARISON TEST \n");
    printf("===== \n");

    for (int i = 0; i < num_sizes; i++) {
        int num_blocks = (100 * MB) / block_sizes[i]; // Total 100MB
        char test_name[100];
        snprintf(test_name, sizeof(test_name), "Sequential Write (Block:
%zu bytes)",
                block_sizes[i]);

        BenchmarkResult result = benchmark_sequential_write(filename,
block_sizes[i], num_blocks);
        print_result(test_name, result);

        unlink(filename);
    }
}

void run_comprehensive_benchmark() {
    const char *seq_file = "sequential_test.dat";

```

```

const char *rand_file = "random_test.dat";
BenchmarkResult result;

printf("\n=====\\n");
printf("COMPREHENSIVE I/O BENCHMARK\\n");
printf("=====\\n");

// Sequential Write
result = benchmark_sequential_write(seq_file, 4096, 10000);
print_result("Sequential Write (4KB blocks)", result);

// Sequential Read
result = benchmark_sequential_read(seq_file, 4096);
print_result("Sequential Read (4KB blocks)", result);

// Random Write
result = benchmark_random_write(rand_file, 4096, 1000);
print_result("Random Write (4KB blocks)", result);

// Random Read
result = benchmark_random_read(rand_file, 4096, 1000);
print_result("Random Read (4KB blocks)", result);

// Cleanup
unlink(seq_file);
unlink(rand_file);
}

int main() {
printf("=== I/O PERFORMANCE BENCHMARK ===\\n");
printf("Starting benchmark suite...\\n");

srand(time(NULL));

run_comprehensive_benchmark();
compare_block_sizes();

printf("\n=====\\n");
printf("Benchmark Complete!\\n");
printf("=====\\n");

return 0;
}

```

Compile dan Jalankan:

```

gcc -o io_benchmark io_benchmark.c
./io_benchmark

```

6.3 Praktik: Asynchronous I/O

Buat file `async_io.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <aio.h>
#include <errno.h>
#include <sys/time.h>

#define NUM_OPERATIONS 10
#define BUFFER_SIZE 4096

void synchronous_io_demo() {
    int fd;
    char buffer[BUFFER_SIZE];
    struct timeval start, end;
    double duration;

    printf("=== Synchronous I/O Demo ===\n");

    gettimeofday(&start, NULL);

    for (int i = 0; i < NUM_OPERATIONS; i++) {
        char filename[50];
        snprintf(filename, sizeof(filename), "sync_file_%d.dat", i);

        fd = open(filename, O_CREAT | O_WRONLY | O_TRUNC, 0644);
        memset(buffer, 'A' + i, BUFFER_SIZE);
        write(fd, buffer, BUFFER_SIZE);
        fsync(fd);
        close(fd);

        printf("Completed operation %d\n", i + 1);
    }

    gettimeofday(&end, NULL);
    duration = (end.tv_sec - start.tv_sec) + (end.tv_usec - start.tv_usec)
    / 1000000.0;

    printf("Total time: %.4f seconds\n", duration);

    // Cleanup
    for (int i = 0; i < NUM_OPERATIONS; i++) {
        char filename[50];
        snprintf(filename, sizeof(filename), "sync_file_%d.dat", i);
        unlink(filename);
    }
}

void asynchronous_io_demo() {
    struct aiocb *aiocb_list[NUM_OPERATIONS];
```

```
char *buffers[NUM_OPERATIONS];
int fds[NUM_OPERATIONS];
struct timeval start, end;
double duration;

printf("\n=== Asynchronous I/O Demo ===\n");

// Initialize
for (int i = 0; i < NUM_OPERATIONS; i++) {
    char filename[50];
    snprintf(filename, sizeof(filename), "async_file_%d.dat", i);

    fds[i] = open(filename, O_CREAT | O_WRONLY | O_TRUNC, 0644);

    buffers[i] = malloc(BUFFER_SIZE);
    memset(buffers[i], 'A' + i, BUFFER_SIZE);

    aiocb_list[i] = malloc(sizeof(struct aiocb));
    memset(aiocb_list[i], 0, sizeof(struct aiocb));

    aiocb_list[i]->aio_fildes = fds[i];
    aiocb_list[i]->aio_buf = buffers[i];
    aiocb_list[i]->aio_nbytes = BUFFER_SIZE;
    aiocb_list[i]->aio_offset = 0;
}

gettimeofday(&start, NULL);

// Submit all I/O operations
for (int i = 0; i < NUM_OPERATIONS; i++) {
    if (aio_write(aiocb_list[i]) == -1) {
        perror("aio_write");
    } else {
        printf("Submitted async operation %d\n", i + 1);
    }
}

// Wait for all operations to complete
for (int i = 0; i < NUM_OPERATIONS; i++) {
    while (aio_error(aiocb_list[i]) == EINPROGRESS) {
        usleep(1000); // Sleep 1ms
    }

    int ret = aio_return(aiocb_list[i]);
    if (ret != -1) {
        printf("Completed operation %d (%d bytes)\n", i + 1, ret);
    }
}

gettimeofday(&end, NULL);
duration = (end.tv_sec - start.tv_sec) + (end.tv_usec - start.tv_usec)
/ 1000000.0;

printf("Total time: %.4f seconds\n", duration);
```

```

// Cleanup
for (int i = 0; i < NUM_OPERATIONS; i++) {
    close(fds[i]);
    free(buffers[i]);
    free(aiocb_list[i]);

    char filename[50];
    snprintf(filename, sizeof(filename), "async_file_%d.dat", i);
    unlink(filename);
}

int main() {
    printf("=== Synchronous vs Asynchronous I/O Comparison ===\n\n");

    synchronous_io_demo();
    asynchronous_io_demo();

    printf("\nNote: Asynchronous I/O allows operations to run in
parallel,\n");
    printf("potentially improving performance for multiple I/O
operations.\n");

    return 0;
}

```

Compile dan Jalankan:

```

gcc -o async_io async_io.c -lrt
./async_io

```

6.4 Praktik: Monitoring dengan perf dan profiling

Buat file `profile_io.sh`:

```

#!/bin/bash

echo "=== I/O Performance Profiling ==="
echo

# Compile program untuk di-profile
gcc -o io_benchmark io_benchmark.c

echo "1. Running iostat monitoring..."
echo "Press Ctrl+C after a few seconds"
echo
iostat -x 1 5

```

```
echo
echo "2. Disk usage statistics"
df -h

echo
echo "3. Running benchmark with time profiling..."
time ./io_benchmark

echo
echo "4. Checking I/O wait time"
vmstat 1 5

echo
echo "Profile complete!"
```

Jalankan:

```
chmod +x profile_io.sh
./profile_io.sh
```

6.5 Praktik: Perbandingan Berbagai Metode I/O

Buat file `io_comparison.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <sys/time.h>

#define FILE_SIZE (10 * 1024 * 1024) // 10 MB
#define BUFFER_SIZE 4096

double get_time_diff(struct timeval start, struct timeval end) {
    return (end.tv_sec - start.tv_sec) + (end.tv_usec - start.tv_usec) /
    1000000.0;
}

void method1_standard_io() {
    FILE *fp;
    char buffer[BUFFER_SIZE];
    struct timeval start, end;

    printf("\n=== Method 1: Standard I/O (fread/fwrite) ===\n");

    // Write
```

```
fp = fopen("test_stdio.dat", "w");
memset(buffer, 'X', BUFFER_SIZE);

gettimeofday(&start, NULL);
for (int i = 0; i < FILE_SIZE / BUFFER_SIZE; i++) {
    fwrite(buffer, 1, BUFFER_SIZE, fp);
}
fclose(fp);
gettimeofday(&end, NULL);

printf("Write time: %.4f seconds\n", get_time_diff(start, end));

// Read
fp = fopen("test_stdio.dat", "r");
gettimeofday(&start, NULL);
while (fread(buffer, 1, BUFFER_SIZE, fp) > 0);
fclose(fp);
gettimeofday(&end, NULL);

printf("Read time: %.4f seconds\n", get_time_diff(start, end));

unlink("test_stdio.dat");
}

void method2_system_calls() {
    int fd;
    char buffer[BUFFER_SIZE];
    struct timeval start, end;

    printf("\n=== Method 2: System Calls (read/write) ===\n");

    // Write
    fd = open("test_syscall.dat", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    memset(buffer, 'X', BUFFER_SIZE);

    gettimeofday(&start, NULL);
    for (int i = 0; i < FILE_SIZE / BUFFER_SIZE; i++) {
        write(fd, buffer, BUFFER_SIZE);
    }
    fsync(fd);
    close(fd);
    gettimeofday(&end, NULL);

    printf("Write time: %.4f seconds\n", get_time_diff(start, end));

    // Read
    fd = open("test_syscall.dat", O_RDONLY);
    gettimeofday(&start, NULL);
    while (read(fd, buffer, BUFFER_SIZE) > 0);
    close(fd);
    gettimeofday(&end, NULL);

    printf("Read time: %.4f seconds\n", get_time_diff(start, end));
}
```

```

    unlink("test_syscall.dat");
}

void method3_memory_mapped() {
    int fd;
    char *mapped;
    struct timeval start, end;

    printf("\n=== Method 3: Memory Mapped I/O (mmap) ===\n");

    // Write
    fd = open("test_mmap.dat", O_CREAT | O_RDWR | O_TRUNC, 0644);
    ftruncate(fd, FILE_SIZE);

    gettimeofday(&start, NULL);
    mapped = mmap(NULL, FILE_SIZE, PROT_WRITE, MAP_SHARED, fd, 0);
    if (mapped != MAP_FAILED) {
        memset(mapped, 'X', FILE_SIZE);
        msync(mapped, FILE_SIZE, MS_SYNC);
        munmap(mapped, FILE_SIZE);
    }
    close(fd);
    gettimeofday(&end, NULL);

    printf("Write time: %.4f seconds\n", get_time_diff(start, end));

    // Read
    fd = open("test_mmap.dat", O_RDONLY);
    gettimeofday(&start, NULL);
    mapped = mmap(NULL, FILE_SIZE, PROT_READ, MAP_PRIVATE, fd, 0);
    if (mapped != MAP_FAILED) {
        volatile char dummy;
        for (int i = 0; i < FILE_SIZE; i += BUFFER_SIZE) {
            dummy = mapped[i]; // Force read
        }
        munmap(mapped, FILE_SIZE);
    }
    close(fd);
    gettimeofday(&end, NULL);

    printf("Read time: %.4f seconds\n", get_time_diff(start, end));

    unlink("test_mmap.dat");
}

void method4_direct_io() {
    int fd;
    void *buffer;
    struct timeval start, end;

    printf("\n=== Method 4: Direct I/O (O_DIRECT) ===\n");

    // Allocate aligned buffer for O_DIRECT
    if (posix_memalign(&buffer, 512, BUFFER_SIZE) != 0) {

```

```

        printf("Failed to allocate aligned buffer\n");
        return;
    }

    memset(buffer, 'X', BUFFER_SIZE);

    // Write
    fd = open("test_direct.dat", O_CREAT | O_WRONLY | O_TRUNC | O_DIRECT,
0644);
    if (fd == -1) {
        printf("O_DIRECT not supported, using O_SYNC instead\n");
        fd = open("test_direct.dat", O_CREAT | O_WRONLY | O_TRUNC |
O_SYNC, 0644);
    }

    gettimeofday(&start, NULL);
    for (int i = 0; i < FILE_SIZE / BUFFER_SIZE; i++) {
        write(fd, buffer, BUFFER_SIZE);
    }
    close(fd);
    gettimeofday(&end, NULL);

    printf("Write time: %.4f seconds\n", get_time_diff(start, end));

    // Read
    fd = open("test_direct.dat", O_RDONLY | O_DIRECT);
    if (fd == -1) {
        fd = open("test_direct.dat", O_RDONLY);
    }

    gettimeofday(&start, NULL);
    while (read(fd, buffer, BUFFER_SIZE) > 0);
    close(fd);
    gettimeofday(&end, NULL);

    printf("Read time: %.4f seconds\n", get_time_diff(start, end));

    free(buffer);
    unlink("test_direct.dat");
}

int main() {
    printf("=== I/O Method Comparison Benchmark ===\n");
    printf("File size: %d MB\n", FILE_SIZE / (1024 * 1024));
    printf("Buffer size: %d KB\n", BUFFER_SIZE / 1024);

    method1_standard_io();
    method2_system_calls();
    method3_memory_mapped();
    method4_direct_io();

    printf("\n=== Summary ===\n");
    printf("Standard I/O: Best for portability and automatic
buffering\n");

```

```
    printf("System Calls: More control, less overhead from buffering  
layer\n");  
    printf("Memory Mapped: Fastest for random access and large files\n");  
    printf("Direct I/O: Bypasses cache, useful for database systems\n");  
  
    return 0;  
}
```

Compile dan Jalankan:

```
gcc -o io_comparison io_comparison.c  
./io_comparison
```

Tugas 6.1:

1. Jalankan `io_benchmark` dan catat hasil throughput dan IOPS
2. Bandingkan performa sequential vs random I/O, jelaskan perbedaannya
3. Jalankan `io_comparison` dan tentukan metode I/O mana yang paling cepat untuk use case Anda
4. Gunakan `iostat` untuk monitoring real-time saat menjalankan benchmark
5. Analisis bottleneck dari hasil monitoring

REFERENSI DAN RESOURCES

Dokumentasi Linux

```
# Man pages yang relevan  
man 2 open  
man 2 read  
man 2 write  
man 2 mmap  
man 7 pipe  
man 1 iostat  
man 1 iotop  
  
# Kernel documentation  
/usr/share/doc/linux-doc/
```

Tools Monitoring

- **iostat**: I/O statistics
- **iotop**: I/O monitoring per process
- **vmstat**: Virtual memory statistics
- **perf**: Performance analysis
- **strace**: System call tracer
- **blktrace**: Block layer I/O tracing

Useful Commands

```
# Cek I/O scheduler
cat /sys/block/sda/queue/scheduler

# Cek disk I/O stats
cat /proc/diskstats

# Monitor real-time I/O
watch -n 1 'cat /proc/diskstats'

# Clear page cache (perlu sudo)
sync && echo 3 > /proc/sys/vm/drop_caches

# Monitor specific process I/O
sudo iotop -p <PID>
```

TROUBLESHOOTING

Problem 1: Permission Denied

```
# Jika tidak bisa akses /proc atau /sys
sudo chmod +r /proc/interrupts
# Atau jalankan program dengan sudo
```

Problem 2: Compilation Errors

```
# Install missing libraries
sudo apt install build-essential libaio-dev

# Jika error dengan aio.h
sudo apt install libaio-dev
```

Problem 3: O_DIRECT Not Supported

```
# O_DIRECT mungkin tidak didukung di filesystem tertentu
# Gunakan O_SYNC sebagai alternatif
```

Problem 4: Low Performance

```
# Check disk scheduler
cat /sys/block/sda/queue/scheduler
```

```
# Change to deadline or noop if needed  
echo deadline | sudo tee /sys/block/sda/queue/scheduler
```